NAME :S.GAYATHERI

REG NO: 192521306

COURSE CODE : CSA0619

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

CO3 - ASSESSMENT TOOL – 02

Q12. Online Gaming Leaderboard using Quick Sort

Aim

To analyze the application of the Quick Sort algorithm in an online gaming leaderboard for efficiently sorting players based on their scores, understand its divide-and-conquer approach, evaluate its performance, identify challenges such as pivot selection and worst-case complexity, and propose suitable optimizations.

Problem Statement

An online gaming platform continuously updates player scores after every match. To display the latest rankings in real time, the system requires a fast and efficient sorting algorithm. Quick Sort is widely used because of its excellent average-case performance and its divide-and-conquer strategy.

Objective

- Sort player scores efficiently.
- Update rankings in real time.
- Reduce leaderboard refresh time.
- Improve sorting performance using optimized pivot selection.
- Analyze time and space complexity.

Working of Divide and Conquer

Divide

Choose a pivot and divide the player scores into two groups.

Example:

Scores:

[450, 700, 300, 850, 600]

Pivot = 600

Left:

450 300

Right:

700 850

Conquer

Sort both groups recursively.

Left:

300 450

Right:

700 850

Combine

Final sorted leaderboard

300 450 600 700 850

Issues in Quick Sort

1. Pivot Selection

Choosing the first or last element repeatedly can produce highly unbalanced partitions.

Example

Already sorted data

100

200

300

400

500

Produces

1 element

4 elements

↓

1 element

3 elements

↓

1 element

2 elements

This results in $O(n^2)$ time.

2. Worst Case Performance

Occurs when

- Array is already sorted.
- Array is reverse sorted.
- Poor pivot selection.
- Many duplicate values.

Worst-case complexity:

$O(n^2)$

3. Recursive Calls

Large datasets increase recursion depth, consuming more memory**.**

Optimization

Randomized Pivot Selection

Instead of selecting the first or last element, randomly choose a pivot.

Advantages

- Avoids worst-case partitions.
- Produces balanced partitions.
- Improves average performance.
- Better suited for dynamic gaming leaderboards.

Algorithm

Step 1: Start.

Step 2: Read the player scores.

Step 3: Select a random pivot.

Step 4: Partition the scores into:

- Less than pivot
- Equal to pivot
- Greater than pivot

Step 5: Recursively sort left and right partitions.

Step 6: Combine all partitions.

Step 7: Display the sorted leaderboard.

Step 8: Stop.

SOURCE CODE

main.py

Share Run

```python
import random

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = random.choice(arr)

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

n = int(input("Enter number of players: "))

scores = []

print("Enter player scores:")

for i in range(n):
    scores.append(int(input()))

sorted_scores = quick_sort(scores)

print("Sorted Leaderboard:")

for score in sorted_scores[::-1]:
    print(score)
```

OUTPUT

Output

```
Enter number of players: 6
Enter player scores:
356
562
852
451
213
651
Sorted Leaderboard (Highest to Lowest):
852
651
562
451
356
213

=== Code Execution Successful ===
```

Complexity Analysis

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

Space Complexity

O(log n)

(Recursion stack in average case)

Result

The **Quick Sort** algorithm successfully sorts player scores for an online gaming leaderboard using the **divide-and-conquer** technique. It provides an average time complexity of **O(n log n)**, making it suitable for real-time ranking systems. By adopting **randomized pivot selection**, the likelihood of worst-case performance is significantly reduced, resulting in faster and more efficient leaderboard updates.